

CCAT（基于协同排序的计算机自适应测试）精读笔记

原文: ccat-collaborative-ranking-cat_14.pdf (NeurIPS 2024, Zirui Liu, Yan Zhuang, Qi Liu 等, USTC & 科大讯飞) · 代码: <https://github.com/bigdata-ustc/CCAT> (已下载到本目录 CCAT/) · 笔记于 2026-06-26

一、解决什么问题

一句话: 以往 CAT 都在拼命把每个学生的能力 θ 估准 (最小化 MSE), 但本文指出——估得准 $\neq$ 排得对。在 GRE、GMAT 这种选拔性高利害考试里, “谁比谁强”的排序往往比“某人能力具体是几分”更重要。

痛点拆开看:

1. 低 MSE 不保证排序对 (论文图 1a)。两个学生能力估计的误差都很小, 但相对大小可能颠倒——能力真高的反而排在后面, 学生会觉得考试不公平。
2. SOTA 也翻车 (图 1b)。准确性最强的 BECAT, 在排序指标上有时只相当于随机选题。
3. 学生之间是“独立、异步”的。传统 CAT 每个学生各考各的、各答各的题, 没法把“全体学生的信息”放在一起来保证彼此排序正确。这是技术上的核心障碍。

本文主张: 与其继续优化“估准”, 不如把 CAT 的目标重新定义为排序一致性 (ranking consistency), 并打破“学生独立”这个假设——引入一批协同学生 (collaborative students) 当锚点, 让所有被测学生都跟这批锚点比, 从而间接比出彼此的高低。

和已有工作比好在哪: (1) 第一个形式化“CAT 排序一致性困境”并给出定义和近似; (2) 给出有理论保证的协同方法 (定理 1: 协同学生够多, 排序一致性概率 $\geq 1 - \delta$); (3) 即插即用——可套在 FSI/KLI/MAAT/NCAT/BECAT 任何选题算法上 (论文里的 X-C 变体), 排序一致性平均涨 5%, 短测试时更明显。

二、问题定义 (输入 / 输出)

符号。- 题库 Q : 每个学生考 T 题, 构成子集 $S \subseteq Q$ 。- 第 j 题两个参数: 区分度 a_j 、难度 b_j (IRT 的, 考前已标定)。- 学生作答 $y_j \in \{0, 1\}$; IRT 能力 θ 。- 真实能力 θ^* : 用学生在整个题库上的完整作答估出来的能力, 当成“真值”。- 测试结果 θ^T : 只用考过的 T 题估出来的能力。- 协同学生: M 名已经答完题库所有题的学生, 能力 θ_c^*, θ_c^T 已知, 当锚点用。

任务的输入/输出 (CCAT 这个框架): - 输入: 被测学生当前已答题及对错 + 协同学生在这些题上的作答 ($M \times |Q|$ 的作答矩阵) + IRT 参数。- 输出: (1) 每一步选哪道题; (2) 测试结束时被测学生的排序分 $\hat{\theta}^T$ (= 在 M 个锚点中排第几的度量)。

目标 (式 3): 让真实能力高的学生测试结果也高, 即最大化 $P(\theta_1^T > \theta_2^T \mid \theta_1^* > \theta_2^*)$ 。

一个具体小例子。学生 A 考了 q_1, q_2, q_3 , 学生 B 考了 q_4, q_5, q_6 ——题都不一样, 没法直接比 A、B 谁强。引入协同学生小明 (题库全答过): - 小明在 q_1, q_2, q_3 上的表现 vs A $\rightarrow$ 小明赢 A; - 小明在 q_4, q_5, q_6 上的表现 vs B $\rightarrow$ 小明输 B。- 结论: B > 小明 > A, 于是 B 排在 A 前面。一个锚点就把两个考不同题的人比了出来。 M 个锚点投票, 排序就稳了。

三、模型结构

CCAT 没有要训练的神经网络 (这是它和 NCAT 的最大区别), 核心就是两个用协同学生信息算出来的公式, 套进标准 CAT 循环 (选题 $\rightarrow$ 作答 $\rightarrow$ 估能力, 重复 T 步)。框架两部分 (论文图 2):

模块 A: 选题部分 (式 7)

不再问“哪道题信息量最大”, 而是问“哪道题最能把被测学生和锚点们的高低分清楚”。- 对每道候选题, 计算它对“区分被测学生 vs 各协同学生”的贡献分 F , 选 F 最大的。- 直觉: 挑那种“比你强的锚点能答对、你答不出”或“比你弱的锚点答不对、你能答对”的题——这种题能把你和锚点的相对位置钉死。

模块 B：能力估计部分（式 8 / 算法 1 第 5 行）

测试结束后，不直接用 IRT 的 θ^T 当结果，而是算被测学生在 M 个锚点里的相对排名 $\hat{\theta}^T$ ，用这个排名当最终成绩。本质上是“协同学生给被测学生投票”。

数据流：被测学生答一题 → 拿到该题答案 + 锚点们在该题的答案 → 更新 F （式 7）选下一题 → 重复 → 最后用式 8 把“我赢了多少个锚点”折算成排序分。

四、关键公式详解（重点）

公式 (1)：IRT 能力估计（地基）

$$\theta^t = \arg \max_{\theta} \sum_{j=1}^t y_j \ln P_j(\theta) + (1 - y_j) \ln(1 - P_j(\theta)), \quad P_j(\theta) = \sigma(a_j(\theta - b_j))$$

- 符号： $P_j(\theta)$ 能力 θ 答对第 j 题的概率；最大化的是已答 t 题的对数似然。- 举例：学生答了 2 题， q_1 ($a = 1, b = 0$) 答对、 q_2 ($a = 1, b = 2$) 答错。似然 $= \ln \sigma(\theta) + \ln(1 - \sigma(\theta - 2))$ 。对 θ 求极值，解出来 $\theta \approx 1.0$ 左右（介于 0 和 2 之间，符合“答对易题、答错难题”的直觉）。- 作用：CCAT 每步都用它更新被测学生的 θ （代码里 `scipy.optimize.root(Likelihood,...)`）。

公式 (5)：把“能力比较”换成“加权作答之差”——全文的关键简化

$$P(\theta^T > \theta_c^T \mid \theta^* > \theta_c^*, S) \Rightarrow \sum_{q_i \in S} R(q_i \mid \theta^* > \theta_c^*), \quad R(q_i \mid \theta^* > \theta_c^*) = a_i P(y_i = 1 \mid \theta^*) P(y_i^c = 0)$$

- 来自引理 1：若两人考同一组题，则 $\theta^T - \theta_c^T$ 与 $\sum_i a_i(y_i - y_i^c)$ 同号。所以“比能力高低”可以直接换成“比加权作答之和”，不用真去解 IRT。- 符号： a_i 区分度（权重）； $P(y_i = 1 \mid \theta^*)$ 被测学生答对的概率； $P(y_i^c = 0)$ 协同学生答错的概率。 R 越大，说明“这题被测生会、锚点不会”，越能证明被测生 > 锚点。- 举例：某题 $a_i = 2$ ，被测生答对概率 0.8，锚点答错概率 0.6 $\rightarrow R = 2 \times 0.8 \times 0.6 = 0.96$ 。换一道锚点很可能答对的题（答错概率 0.1） $\rightarrow R = 2 \times 0.8 \times 0.1 = 0.16$ ，贡献小得多。所以模型偏爱前一种题。- 作用：把“最大化排序一致性概率”这个难算的目标，变成一个可逐题累加的、好优化的分数。

公式 (7)：CCAT 选题准则（核心，代码 `get_ccat`）

$$q_t = \arg \max_{q \in Q \setminus S_{t-1}} P(\theta^* < \theta_c^*) R(q \mid \theta^* < \theta_c^*) + P(\theta^* > \theta_c^*) R(q \mid \theta^* > \theta_c^*)$$

- 两项对称：第一项管“锚点比我强”的情况（找锚点会、我不会的题），第二项管“锚点比我弱”的情况（找锚点不会、我会的题）。两种题都能把我和锚点拉开。- $P(\theta^* \geq \theta_c^*)$ 是“我和这个锚点谁强”的当前估计概率，用已答题的加权差经 sigmoid 得到。- 举例：候选题对某个“可能比我强”的锚点： $a = 2$ 、我答对概率 $P_t = 0.3$ 、锚点答对 (`train_label=1`)。则该题对第一项的贡献 $= a \cdot (1 - P_t) \cdot \text{train_label} = 2 \times 0.7 \times 1 = 1.4$ （我大概率答错、锚点答对 $\rightarrow$ 能确认锚点 > 我）。对所有锚点求和后取最大，就是下一题。- 代码印证 (`selection_strategy.py:39-44`):

```
python Pt = self.P(theta) # 被测生各题答对概率 P_t Ptheta = sigmoid((ga*(train_label - test_label[stu])*selected).sum(-1)) # 约 = P(theta* < theta*_c), 每个锚点一个 F = ga*(Ptheta*(1-Pt)*train_label + (1-Ptheta)*Pt*(1-train_label)) # 正是式 (7) 两项 return F.sum(0) # 对 M 个锚点求和  $\rightarrow$  每道题一个分 ga= 区分度 a, train_label= 锚点作答  $y^c$ , 两行  $Ptheta*(1-Pt)*train\_label$  和  $(1-Ptheta)*Pt*(1-train\_label)$  一对对应式 7 的两项。
```

公式 (8)：协同能力估计 = 锚点投票

$$\hat{\theta}^T = \mathbb{E} \left[\mathbb{I} \left(\sum_{i \in S} a_i(y_i - y_i^c) > 0 \right) \right] \approx \frac{1}{M} \sum_{i=1}^M \sigma \left(\sum_{t=1}^T a_i(y_{it}^c - y_t) \right)$$

- 符号： $\mathbb{I}$ 指示函数（赢了记 1）；对 M 个锚点取平均，就是“我赢过的锚点比例”。- 举例： $M = 5$ 个锚点，被测生赢了其中 3 个 ($\sum a_i(y_i - y_i^c) > 0$ 对 3 个锚点成立) $\rightarrow \hat{\theta}^T = 3/5 = 0.6$ 。另一个学生赢了 4 个 $\rightarrow 0.8 > 0.6$ ，于是后者排前面。用这个 $\hat{\theta}^T$ 当成绩，天然就是排序量。- 作用：把“绝对能力分”换成“相对排名分”，直接对齐排序目标。

定理 1：需要多少锚点才够

$$M > \frac{\ln \frac{1}{\delta}}{2(\zeta_1 - \zeta_2)^2} \Rightarrow \text{排序一致性概率} \geq 1 - \delta; \quad \text{有排名误差} \rho \text{ 时: } M > \frac{\ln \frac{1}{\delta}}{2(1 - 2\rho)^2(\zeta_1 - \zeta_2)^2}$$

- 符号: ζ_1, ζ_2 是 A、B 各自胜过单个锚点的概率; $\zeta_1 - \zeta_2$ 越大说明两人越好区分, 需要的锚点越少; ρ 是单次比较出错的概率。- 举例: 想要 95% 把握 ($\delta = 0.05$), 且 $\zeta_1 - \zeta_2 = 0.2 \rightarrow M > \frac{\ln 20}{2 \times 0.04} = \frac{3.0}{0.08} \approx 37$ 个锚点就够。若每次比较有 $\rho = 0.1$ 的误差, 分母多乘 $(1 - 0.2)^2 = 0.64 \rightarrow M > 58$ 。- 作用: 用 Hoeffding 不等式给出“锚点数量 $\rightarrow$ 排序可靠度”的硬保证, 这是本文相对 NCAT/BECAT 最大的理论亮点。而且式 7 的选题正是在压低 ρ , 理论与算法闭环。

五、代码实现对照

下载到 CCAT/, 是纯 NumPy/SciPy/PyTorch 实现, 无需训练大模型 (NCAT 基线才需要预训练权重)。

论文内容	代码位置
主流程 (选题 $\rightarrow$ 作答 $\rightarrow$ 估能力, 循环 20 步)	selection_strategy.py $\rightarrow$ MCMC_Selection.get_question / GD_Selection.get_question (for i in range(20))
式 (7) CCAT 选题	get_ccat(): $F = ga * (P_{\theta} * (1 - P_t) * \text{train_label} + (1 - P_{\theta}) * P_t * (1 - \text{train_label}))$
式 (1) IRT 能力估计	MCMC 版用 scipy.optimize.root(Likelihood,...); GD 版用 self.irt.optim(...) (IRT.py)
协同能力估计 (式 8 投票)	在 main.py 里对选完的题用 sigmoid($\sum a_i(y^c - y)$) 求排名分
基线 FSI/KLI/MAAT/BECAT/NCAT	同文件内 get_Fisher / get_kli / get_maat / get_becat / get_ncat
IRT 参数 (GD/MCMC 两种估计)	IRT.py、data/mcmc.py
数据预处理、4:1 划分协同/被测	data/prepare_data.py、dataset.py

关键对应: train_label = 协同学生作答矩阵 ($M \times |Q|$), $\text{test_label}[\text{stu}]$ = 被测学生作答, selected = 已选题掩码, ga/be = IRT 的 a/b 。代码默认走 cuda。

六、小结

- 贡献: 把 CAT 目标从“估准能力”重定义为“排序一致性”; 用协同学生当锚点, 给出选题 (式 7) + 估计 (式 8) 两件套, 且有理论保证 (定理 1); 方法即插即用, 能嫁接到任何已有选题算法。
- 效果: NIPS-EDU、JUNYI 两数据集 $\times$ GD/MCMC 两种 IRT 估计上, 排序一致性 (Kendall) 全面领先, 平均 +5%, 题目越少优势越大; 同时 ACC/AUC 不掉 (准确性没牺牲)。附带结论: MCMC 估 IRT 比 GD 更稳、更适合排序场景。
- 优点: 无需训练、可解释 (投票机制)、有理论支撑、通用。
- 局限:
 1. 长测试时优势缩小 (论文结论自己承认), 因为题一多 IRT 本身就估得准, 协同的边际价值下降;
 2. 依赖“有一大批已答完整题库的协同学生”, 冷启动/小数据场景难构建锚点;
 3. 空间复杂度高 (要存所有协同学生作答), 选题复杂度 $O(|Q|TMN)$ 随锚点数 M 线性增长;
 4. 沿用 IRT 单维能力与“能力测试期间不变”假设, 多维知识/有学习遗忘的场景未必适配。